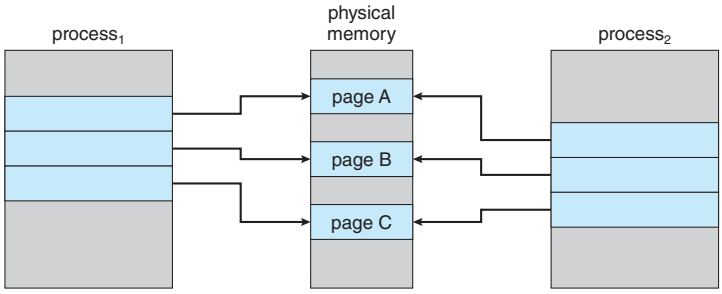
همان‌گونه که در بخش ۹-۵-۳ بیان شد، سیستم‌عامل‌های موبایل نوعاً از مبادله پشتیبانی نمی‌کنند. در عوض، این سیستم‌ها صفحه را از سیستم‌فایل تقاضا می‌کنند و در صورتی که حافظه محدود شود، صفحات فقط‌خواندنی (همچون کد) را از برنامه‌ها پس می‌گیرند. این قبیل داده‌ها در صورتی که بعداً لازم شوند، می‌توانند از سیستم‌فایل تقاضا شوند. در iOS، صفحات ناشناس حافظه هرگز از یک برنامه پس‌گرفته نمی‌شوند مگر این که برنامه خاتمه بپذیرد یا آشکارا حافظه را رها کند. ما در بخش ۱۰-۷، حافظه‌ی فشرده را پوشش می‌دهیم، یک روش دیگر رایجی که برای مبادله در سیستم‌های موبایل استفاده می‌شود.

## ۱۰-۳ کپی-موقع-نوشتن

در بخش ۱۰-۲ نشان دادیم که یک پردازه چگونه می‌تواند به سرعت با صفحه‌بندی نیازی، صفحه‌ی شامل اولین دستورالعمل خود را بار کرده و کار خود را شروع کند. با این وجود، ایجاد پردازه با استفاده از فراخوانی سیستمی fork() ممکن است در شروع با استفاده از تکنیکی مشابه اشتراک صفحه (که در بخش ۹-۳-۴ پوشش داده شد) الزام صفحه‌بندی نیازی را دور بزند. این تکنیک، ایجاد پردازه را سرعت بخشیده و تعداد صفحات جدیدی را که بایستی در ایجاد پردازه‌ی جدید تخصیص یابند به حداقل می‌رساند.

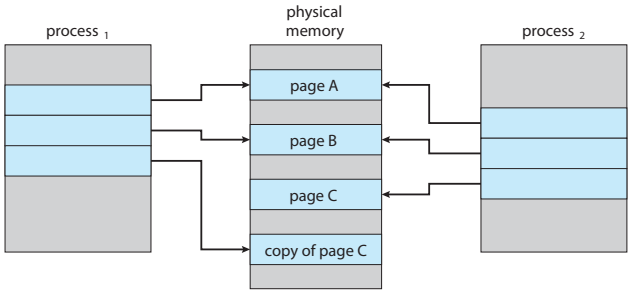
به یاد دارید که فراخوانی سیستمی fork() یک پردازه‌ی فرزند ایجاد می‌کند که کپی والدش می‌باشد. در سابق، fork() با ایجاد یک کپی از فضای آدرس والد برای فرزند، با تکثیر صفحات متعلق به والد کار می‌کرد. با این وجود، نظر به این که بسیاری از پردازه‌های فرزند بعد از ایجاد خود بلافاصله فراخوانی سیستمی exec() را احضار می‌کنند، ضرورتی برای کپی فضای آدرس والد وجود ندارد. به جای آن می‌توانیم از تکنیکی معروف به **کپی-موقع-نوشتن**[1] استفاده کنیم که به پردازه‌های والد و فرزند اجازه می‌دهد تا صفحات یکسان را از ابتدا به اشتراک بگذارند. این صفحات مشترک، صفحات کپی-موقع-نوشتن علامت می‌خورند، به این معنی که اگر هر یک از پردازه‌ها بخواهند صفحه‌ی مشترکی را بنویسند، صفحه‌ی مشترک را کپی می‌کنند. کپی-موقع-نوشتن در شکل‌های ۱۰-۷ و ۱۰-۸ نشان داده می‌شود که محتویات حافظه‌ی فیزیکی را، قبل و پس از تغییر صفحه‌ی C توسط $process_1$ نشان می‌دهند.




شکل ۱۰-۷ قبل از این که $process_1$، صفحه‌ی C را تغییر بدهد.

1. Copy-on-write

برای مثال، فرض کنید که پردازه‌ی فرزند بخواهد صفحه‌ای شامل بخش‌هایی از پشته را تغییر بدهد که جزء صفحاتی است که برای کپی-موقع-نوشتن انتخاب شده است. سیستم‌عامل یک قاب از لیست قاب آزاد بدست آورده و کپی جدیدی از این صفحه می‌گیرد و به فضای آدرس پردازه‌ی فرزند نگاشت می‌دهد. پس از آن، پردازه‌ی فرزند، کپی صفحه‌ی خود را تغییر داده و کاری به صفحه‌ی متعلق به پردازه‌ی والد ندارد. روشن است که در صورت استفاده از تکنیک کپی-موقع-نوشتن، تنها صفحاتی که توسط پردازه‌ها تغییر می‌یابند کپی می‌گردند؛ تمامی صفحات بدون تغییر می‌توانند توسط پردازه‌های والد و فرزند به صورت مشترک استفاده شوند. همین‌طور توجه داشته باشید که فقط صفحاتی که امکان تغییر دارند لازم است به صورت کپی-موقع-نوشتن علامت بخورند. صفحاتی که نمی‌توانند تغییر یابند (صفحاتی که شامل کد اجرایی هستند) می‌توانند توسط والد و فرزند به اشتراک گذاشته شوند. کپی-موقع-نوشتن یک تکنیک رایجی است که توسط سیستم‌عامل‌های متعدد، اعم از ویندوز، لینوکس و macOS استفاده می‌شود.




شکل ۱۰-۸ بعد از تغییر صفحه‌ی C توسط process1.

نسخه‌های متعددی از یونیکس (اعم از لینوکس، macOS، و یونیکس BSD)، گونه‌ای از فراخوانی سیستمی fork()—vfork() (مشتق از **virtual memory fork**)—را فراهم می‌کنند که عملیات آن در کپی-موقع-نوشتن با fork() فرق دارد. پردازه‌ی والد با vfork() معلق می‌شود و پردازه‌ی فرزند از فضای آدرس پردازه‌ی والد استفاده می‌کند. چون vfork() از کپی-موقع-نوشتن استفاده نمی‌کند، هرگونه تغییری که توسط پردازه‌ی فرزند بر روی صفحات فضای آدرس والد انجام بگیرد، والد بلافاصله با از سرگیری اجرا، آنها را خواهد دید. بنابراین، بایستی در استفاده از vfork() احتیاط کامل صورت گیرد تا این اطمینان حاصل شود که پردازه‌ی فرزند، فضای آدرس والد را تغییر نمی‌دهد. بهتر است زمانی از vfork() استفاده شود که پردازه‌ی فرزند بلافاصله بعد از ایجاد خود، exec() را فراخوانی می‌کند. چون هیچ‌گونه کپی صفحه‌ای صورت نمی‌گیرد، vfork() در ایجاد پردازه یک روش کاملاً کارآمد به حساب می‌آید و بعضاً در پیاده‌سازی رابط پوسته‌های خط فرمان یونیکس استفاده می‌شود.

## ۱۰-۴ جایگزینی صفحه

در بحث اخیری که از نرخ خطای صفحه داشتیم، فرض ما بر این بود که هر خطای صفحه حداکثر یکبار اتفاق می‌افتد و آن هم زمانی است که صفحه برای اولین بار رجوع می‌شود. با این

وجود، این نمایش صحت محضی ندارد. اگر پردازه‌ای با ده صفحه فقط نیمی از آنها را استفاده کند، در این صورت صفحه‌بندی نیازی در I/O لازم برای بارگذاری پنج صفحه‌ای که هرگز استفاده نخواهند شد، صرفه‌جویی می‌کند. همین‌طور، با اجرای دو برابر تعداد پردازه‌های فعلی می‌توانیم درجه‌ی چندبرنامگی خود را افزایش دهیم. بنابراین، اگر چهل قاب داشته باشیم، می‌توانیم به جای اجرای چهار پردازه که برای هر یک از پردازه‌ها ده قاب (که پنج تا از آنها هرگز استفاده نخواهند شد) تخصیص داده می‌شود، تعداد هشت پردازه را اجرا کنیم.

اگر درجه‌ی چندبرنامگی خود را افزایش دهیم، حافظه را **ورای آن تخصیص می‌دهیم**[1]. اگر شش پردازه را اجرا کنیم که اندازه‌ی هر یک از آنها ده قاب است، اما به واقع فقط از پنج قاب استفاده می‌کنند، پردازنده‌ای با بهره‌وری و بازده بالاتر به همراه ده قاب یدک هم خواهیم داشت. با این وجود، هر یک از این پردازه‌ها ممکن است به خاطر مجموعه‌ای از داده‌های خاص به طور ناگهانی بخواهند تمامی ده صفحه‌ی خود را استفاده نمایند که در نتیجه، هر شصت قاب مورد نیاز خواهند بود، حال آنکه موجودی ما فقط چهل قاب است.

به علاوه، در نظر داشته باشید که حافظه‌ی سیستم فقط برای نگه‌داری صفحات برنامه استفاده نمی‌شود. بافرهای I/O نیز هستند که حجم قابل توجهی از حافظه را مصرف می‌کنند. با این میزان استفاده از حافظه، نقش الگوریتم‌های تخصیص فضای حافظه پر رنگ‌تر می‌شود. تصمیم‌گیری این که چه مقدار از حافظه برای I/O و چه مقدار آن برای صفحات برنامه تخصیص یابد، چالش مهمی است. برخی سیستم‌ها درصد ثابتی از حافظه را برای بافرهای I/O کنار می‌گذارند، در حالی که سیستم‌های دیگری هم هستند که به هر دوی پردازه‌ها و زیرسیستم I/O اجازه می‌دهند تا برای گرفتن کل حافظه‌ی سیستم با هم به رقابت بپردازند. بخش ۱۴-۶، ارتباط جمعی میان بافرهای I/O و تکنیک‌های حافظه‌ی مجازی را بحث می‌کند.

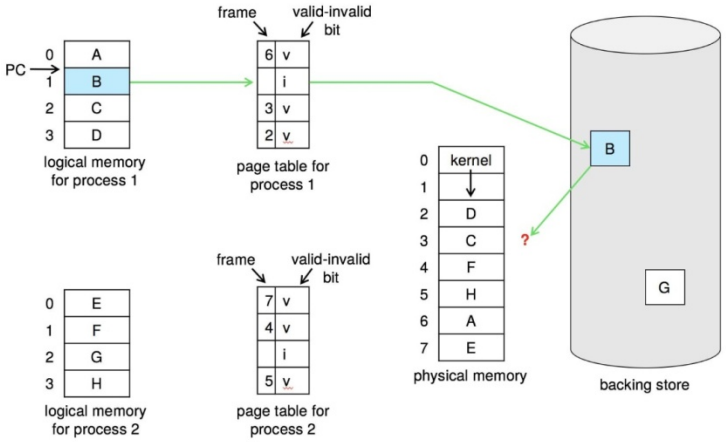
تخصیص ورای حافظه، خود را به این صورت نشان می‌دهد. در حالی که پردازه‌ای از کاربر اجرا می‌شود، یک خطای صفحه رخ می‌دهد. سیستم‌عامل، محل صفحه‌ی مطلوب را در حافظه‌ی ثانویه می‌یابد، اما در ادامه، متوجه می‌شود که هیچ قابی در لیست قاب آزاد نیست؛ تمام حافظه در حال استفاده است. این وضعیت در شکل ۱۰-۹ نمایش داده می‌شود که واقعیت این که هیچ قاب آزادی وجود ندارد با علامت سؤال به‌تصویر کشیده می‌شود.

در اینجا، سیستم‌عامل چندین انتخاب در اختیار دارد. می‌تواند پردازه را خاتمه بدهد. اما، سیستم‌عامل با صفحه‌بندی نیازی می‌خواست بهره‌وری و بازده سیستم کامپیوتر را بهبود ببخشد. کاربران هم نباید متوجه شوند که پردازه‌های آنها در یک سیستم صفحه‌بندی شده اجرا می‌شوند—منطقاً، بایستی صفحه‌بندی برای کاربر شفاف باشد. لذا، این گزینه بهترین انتخاب نیست.

سیستم‌عامل به جای این کار می‌تواند از مبادله‌ی استاندارد استفاده کرده و یک پردازه را به بیرون حافظه مبادله کند و قاب‌های در اختیار آن پردازه را آزاد کرده و سطح چندبرنامگی را کاهش دهد. با این وجود، همان‌گونه که در بخش ۹-۵ بحث شد، مبادله‌ی استاندارد به‌دلیل سربار کپی کامل پردازه‌ها میان حافظه و فضای مبادله، دیگر توسط اکثر سیستم‌عامل‌ها استفاده

1. Over-allocating

نمی‌شود. اکثر سیستم‌عامل‌ها هم‌اکنون صفحات مبادله را با **جایگزینی صفحه** ترکیب می‌کنند، تکنیکی که به تفصیل در باقیمانده‌ی همین بخش بیان می‌کنیم.




شکل ۱۰-۹ لزوم جایگزینی صفحه.

### ۱۰-۴-۱ جایگزینی صفحه‌ی پایه‌ای

جایگزینی صفحه این رویکرد را دنبال می‌کند. اگر قاب آزادی نباشد، یکی را که فعلاً استفاده ندارد، پیدا کرده و آزاد می‌کنیم. به منظور آزادکردن یک قاب می‌توانیم محتویات آن را در فضای مبادله نوشته و جدول صفحه (و سایر جداول مرتبط) را تغییر بدهیم تا نشان بدهد که آن صفحه دیگر در حافظه نیست (شکل ۱۰-۱۰). اکنون می‌توانیم از قاب خالی برای نگه‌داری صفحه‌ای که پردازه به خاطر آن خطا داده است استفاده کنیم. می‌توانیم روال سرویس خطای صفحه را به صورت زیر تغییر بدهیم تا جایگزینی صفحه نیز لحاظ شود:

۱- مکان صفحه‌ی مطلوب را در حافظه‌ی ثانویه پیدا کن.

۲- یک قاب آزاد پیدا کن:

الف) اگر قاب آزادی وجود داشت، از آن استفاده کن.

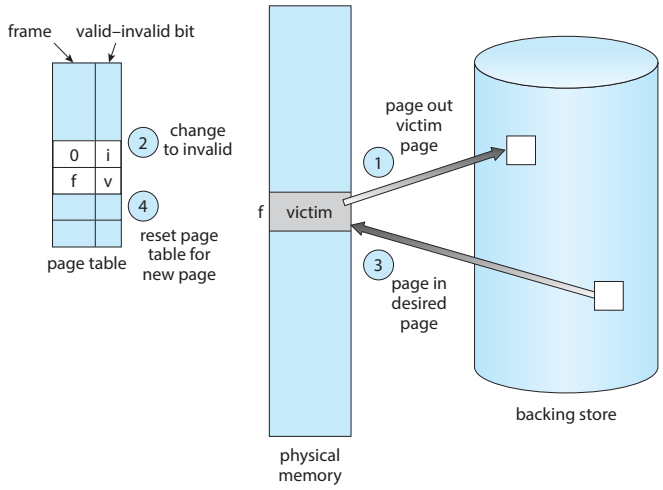
ب) اگر قاب آزادی نباشد، از یک الگوریتم جایگزینی صفحه برای انتخاب یک **قاب قربانی** استفاده کن.

ج) قاب قربانی را روی حافظه‌ی ثانویه بنویس؛ متعاقب آن، جداول قاب و صفحه را تغییر بده.

۳- صفحه‌ی مطلوب را داخل قابی که به تازگی آزاد شده است بخوان؛ جداول صفحه و قاب را تغییر بده.

۴- پردازه را از جایی که خطای صفحه رخ داد، از سر بگیر.

۵- توجه کنید که اگر هیچ یک از قاب‌ها آزاد نباشند، دو انتقال صفحه (یکی برای مبادله‌ی صفحه به بیرون و دیگری برای مبادله‌ی صفحه به داخل) لازمند. با این وضعیت، زمان سرویس خطای صفحه دو برابر می‌شود و متعاقب آن نیز زمان دسترسی مؤثر افزایش می‌یابد.




شکل ۱۰-۱۰ جایگزینی صفحه.

این سربار را می‌توانیم با استفاده از یک **بیت تغییر**[1] (یا **بیت کثیف**[2]) کاهش بدهیم. زمانی که از این طرح استفاده می‌شود، برای هر صفحه یا قاب، یک بیت تغییر سخت‌افزاری منتسب می‌شود. هر بار که یک کلمه یا بایتی از یک صفحه نوشته شود بیت تغییر آن توسط سخت‌افزار یک می‌شود تا نشان بدهد که صفحه تغییر یافته است. زمانی که یک صفحه را برای جایگزینی انتخاب می‌کنیم، بیت تغییر آن را مورد بررسی قرار می‌دهیم. اگر بیت یک باشد متوجه می‌شویم که این صفحه از زمانی که از حافظه‌ی ثانویه خوانده شده، تغییر یافته است. در این حالت، بایستی صفحه را در حافظه‌ی ثانویه بنویسیم. اما، اگر بیت تغییر یک نباشد، صفحه از زمانی که داخل حافظه خوانده شده، تغییری نداشته است. در این حالت، نیازی به نوشتن صفحه‌ی حافظه در دیسک نیست: چون از قبل در آنجا وجود دارد. این تکنیک برای صفحات فقط‌خواندنی (برای مثال، صفحات حاوی کد دودویی) نیز قابل اعمال است. این قبیل صفحات نمی‌توانند تغییر یابند، لذا، در مواقع لازم می‌توان آنها را بیرون انداخت. این طرح می‌تواند زمان لازم را برای سرویس خطای صفحه به صورت قابل توجهی کاهش دهد، چون اگر صفحه تغییر نکرده باشد زمان I/O به نصف کاهش می‌یابد.

1. Modify bit
2. Dirty bit

جایگزینی صفحه، اساس صفحه‌بندی نیازی است. همین‌طور، تفکیک میان حافظه‌ی منطقی و حافظه‌ی فیزیکی را کامل می‌کند. با این راهکار می‌توان با یک حافظه‌ی فیزیکی کوچکتر، حافظه‌ی مجازی عظیمی برای برنامه‌نویسان فراهم کرد. با نبود صفحه‌بندی نیازی، آدرس‌های منطقی به آدرس‌های فیزیکی نگاشته می‌شوند و دو مجموعه‌ی آدرس می‌توانند متفاوت باشند. با این همه، باز هم بایستی تمامی صفحات پردازه در حافظه‌ی فیزیکی قرار بگیرند. با صفحه‌بندی نیازی، دیگر اندازه‌ی فضای آدرس منطقی به حافظه‌ی فیزیکی محدود نمی‌شود. اگر پردازه‌ای با بیست صفحه داشته باشیم، به سادگی می‌توانیم آن را با بهره‌گیری از صفحه‌بندی نیازی در ده قاب اجرا کرده و در مواقع لازم از یک الگوریتم جایگزینی صفحه برای یافتن یک قاب آزاد استفاده کنیم. اگر صفحه‌ای که تغییریافته جایگزین می‌شود، محتویات آن در حافظه‌ی ثانویه کپی می‌گردد. مراجعه‌ی بعدی آن صفحه باعث خطای صفحه خواهد شد. در اینجا، احتمالاً صفحه با جایگزینی صفحه‌ی دیگری از پردازه، باز هم به حافظه آورده می‌شود.

به منظور پیاده‌سازی صفحه‌بندی نیازی بایستی دو مسأله‌ی اصلی را حل کنیم: باید یک **الگوریتم تخصیص قاب** و یک **الگوریتم جایگزینی صفحه** طراحی کنیم. یعنی، اگر قرار باشد پردازه‌های متعددی در داخل حافظه داشته باشیم، بایستی در مورد تعداد قاب‌های تخصیصی به هر یک از پردازه‌ها تصمیم‌گیری کنیم و زمانی که جایگزینی صفحه لازم می‌شود، بایستی قاب‌هایی را برای جایگزینی انتخاب کنیم. طراحی الگوریتم‌های مناسب برای حل این مسائل اهمیت دارد، چرا که I/O حافظه‌ی ثانویه بسیار گران است. حتی بهبودی اندکی در روش‌های صفحه‌بندی نیازی منجر به منافع زیادی در کارایی سیستم می‌گردد.

الگوریتم‌های جایگزینی صفحه‌ی متفاوت بسیاری وجود دارند. هر سیستم‌عامل احتمالاً طرح جایگزینی صفحه‌ی خاص خود را لازم خواهد داشت. چگونه یک الگوریتم جایگزینی صفحه‌ی خاصی را انتخاب کنیم؟ در حالت کلی، الگوریتمی مطلوب ماست که نرخ خطای صفحه‌ی آن پایین‌ترین باشد.

یک الگوریتم را با اجرای آن بر روی رشته‌ی خاصی از مراجعات حافظه و محاسبه‌ی تعداد خطاهای صفحه ارزیابی می‌کنیم. رشته‌ی مراجعات حافظه یک **رشته‌ی مراجعه**[1] نامیده می‌شود. می‌توانیم رشته‌های مراجعه را هوشمندانه (برای مثال، با استفاده از یک مولد اعداد تصادفی) تولید کنیم یا می‌توانیم یک سیستم موجود را ردگیری کرده و آدرس هر یک از مراجعات حافظه را ثبت نماییم. با روش دوم می‌توانیم داده‌های بسیار زیادی تولید کنیم (با مرتبه‌ای از ۱ میلیون آدرس در ثانیه). برای تقلیل تعداد این داده‌ها از دو واقعیت بهره می‌گیریم.

اولاً، با اندازه‌ی صفحاتی که در اختیار داریم (و در حالت کلی، اندازه‌ی صفحه ثابت سخت‌افزار یا سیستم است)، به جای کل آدرس، فقط به شماره‌ی صفحه نیاز داریم. ثانیاً، اگر مراجعه‌ای به صفحه‌ی p داشته باشیم، در این صورت مراجعات *آنی* بعدی p، هرگز خطای صفحه نخواهند داشت. صفحه‌ی p بعد از اولین مراجعه در حافظه خواهد بود، لذا مراجعات آنی بعدی صفحه، خطای صفحه نخواهند داد.

1. Reference string

برای مثال، اگر پردازه‌ی خاصی را ردگیری کنیم، ممکن است دنباله‌ی آدرس‌های زیر را ثبت کنیم:

0100, 0432, 0101, 0612, 0102, 0103, 0104, 0101, 0611, 0102, 0103, 0104, 0101, 0610, 0102, 0103, 0104, 0101, 0609, 0102, 0105

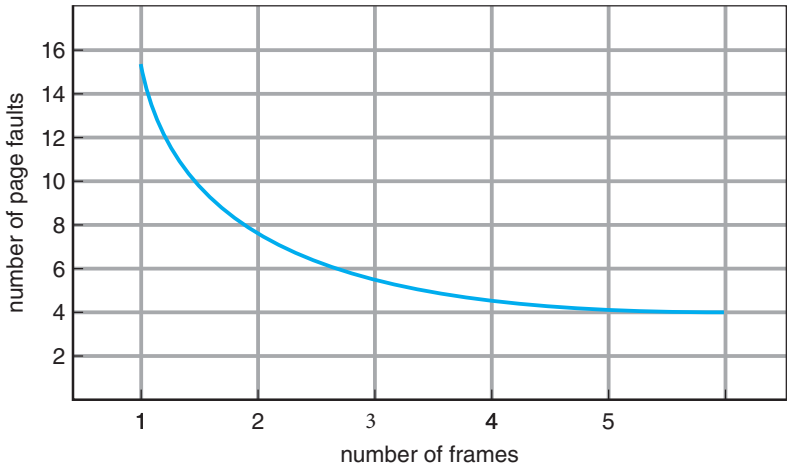
این دنباله با داشتن صفحات ۱۰۰ بایتی می‌تواند به رشته‌ی مراجعه‌ی زیر کاهش یابد:

1, 4, 1, 6, 1, 6, 1, 6, 1, 6, 1

به منظور تعیین تعداد خطاهای صفحه برای یک رشته‌ی مراجعه‌ی خاص با یک الگوریتم جایگزینی صفحه لازم است که تعداد قاب‌های موجود را نیز بدانیم. روشن است که تعداد خطاهای صفحه با افزایش تعداد قاب‌های موجود کاهش می‌یابد. برای مثال، برای رشته‌ی مراجعه که پیشتر دیدیم، اگر به تعداد سه یا بیشتر قاب داشته باشیم، فقط سه مورد خطای صفحه خواهیم داشت که آن هم زمانی است که اولین مراجعه به هر یک از صفحات صورت می‌گیرد. در مقایسه، با داشتن فقط یک قاب، برای هر مراجعه یک جایگزینی صفحه خواهیم داشت که پیامد آن یازده خطا خواهد بود. در حالت کلی، منحنی مورد انتظار ما به شکل ۱۰-۱۱ خواهد بود. تعداد خطاهای صفحه با افزایش تعداد قاب‌ها به یک سطح حداقلی افت می‌کند. البته، تعداد قاب‌ها با افزودن حافظه‌ی فیزیکی افزایش می‌یابد.

در ادامه، چندین الگوریتم جایگزینی صفحه را نشان می‌دهیم. برای انجام این کار از رشته‌ی مراجعات زیر برای حافظه‌ای که سه قاب دارد بهره می‌گیریم:

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1




شکل ۱۰-۱۱ نمودار خطاهای صفحه در مقایسه با تعداد قاب‌ها.

### ۱۰-۴-۲ جایگزینی صفحه‌ی FIFO

ساده‌ترین الگوریتم جایگزینی صفحه، الگوریتم اولین-ورود، اولین-خروج (FIFO) است.